

# Project 02

## Intelligent Systems II

Mário Antunes

2026

### Projects

Form groups of one or two students and select **one** of the following projects. All projects will be hosted on **GitHub**, by creating a **personal or team fork** of the official repository.

The repository must contain all relevant scripts, configuration files, and a comprehensive `README.md`. Your `README.md` serves as your project report and must include instructions on how to run your agent, details regarding your solution's architecture (including state representations, model architectures, and reward functions if applicable), and an evaluation of its performance (e.g., training logs, convergence graphs, hyperparameter selection, and evaluation statistics).

This is a project to develop autonomous agents using Machine Learning (ML), Deep Learning (DL), Reinforcement Learning (RL), or Agentic methodologies. The deadline for submission in the e-learning platform is **22/06/2026**.

Do not forget to contact your professor with any questions. Further instructions may be added.

### Grading

The project will be evaluated based on the following criteria:

| Criteria        | Description                                                   | Weight |
|-----------------|---------------------------------------------------------------|--------|
| <b>Solution</b> | Effectiveness and performance of the ML/DL/RL/Agentic agent.  | 0.30   |
| <b>Code</b>     | Readability, modularity, training design, and best practices. | 0.20   |

| Criteria             | Description                                                | Weight |
|----------------------|------------------------------------------------------------|--------|
| <b>Repository</b>    | Organization of files, checkpoints, and configuration.     | 0.20   |
| <b>Complexity</b>    | Complexity of model design and reward function creativity. | 0.15   |
| <b>Report</b>        | Clarity of training curves and performance evaluation.     | 0.10   |
| <b>Contributions</b> | Finding bugs, fixing code, and submitting Pull Requests.   | 0.05   |

## Topics

The following projects are ordered by difficulty, from lowest to highest. You must select **only one**.

### 1. SI2 - Frogger

- **Repository Link:** [si2-frogger](#)
- **Description:** SI2-Frogger is a real-time simulation platform designed to navigate a frog agent across multiple busy traffic lanes to safely cross checkpoints. The game utilizes a Python WebSocket backend and an HTML5 Canvas frontend. The state is represented dynamically, listing the frog coordinates, remaining lives, score, and the list of moving obstacles (cars with varying positions, widths, and speeds in northbound and southbound lanes).
- **Objective:** Develop an autonomous agent using ML, DL, RL (e.g., Q-Learning, DQN, or PPO), or agentic frameworks to learn when to move NORTH, SOUTH, EAST, or WEST to safely navigate the traffic, hit mid-point and final checkpoints, score points, and maximize surviving laps.

### 2. SI2 - Space Invaders

- **Repository Link:** [si2-space-invaders](#)
- **Description:** SI2-Space-Invaders is a real-time, grid-based shooter where the player controls a ship at the bottom of the screen. The threat comes from rows of hostile aliens performing figure-8 motions, with occasional aliens diving down towards the player. The state includes player location, active lasers fired, and positions of all active aliens.
- **Objective:** Build an autonomous agent using ML, DL, RL, or agentic paradigms to move laterally (WEST/EAST) and shoot strategically to

eliminate waves of invaders while avoiding collisions with diving aliens or letting them reach the bottom.

### 3. SI2 - Breakout

- **Repository Link:** [si2-breakout](#)
  - **Description:** SI2-Breakout is a simulation of the classic brick-breaking game. It runs in a continuous coordinate space (600x400) where a ball bounces off walls and a player-controlled paddle. The state lists the ball's position, radius, velocity, the paddle's width and position, and a list of active bricks.
  - **Objective:** Implement a machine learning, deep learning, or reinforcement learning agent (e.g., using continuous action representations, policy gradients, or DQN) to move the paddle (WEST/EAST) to keep the ball in play, bounce it off the paddle, and clear all brick columns to achieve high scores.
- 

## Workflow & Submission Instructions

Here are detailed instructions to set up your repository and submit your work.

### 1. Create a Fork

1. Go to the repository link for your chosen project (listed in the Topics section above).
2. Click the **Fork** button in the top-right corner of the GitHub interface.
3. Choose your personal account or team organization as the destination.
4. Clone your new fork to your local machine:

```
git clone <URL_OF_YOUR_FORK>
```

### 2. Upstream Contributions (Bonus)

If you identify any bugs, performance issues, or typos in the base simulation code: 1. Fix them in a separate branch on your fork. 2. Submit a **Pull Request** to the original upstream repository. 3. This active contribution to the base class simulation code will be recognized and awarded a bonus under the **Contributions** grading criterion.

### 3. Submission

When submitting your project on the e-learning platform, ensure you provide: 1. The **URL link to your team's GitHub fork**. 2. A brief note indicating the names and student numbers of the group members.